Zerbitzu eta Prozesuen Programazioa

Hariekin Ariketak (Multithreading eta Swing)

EDUKIA

0. Sarrera 1

1. Oinarri Teorikoak eta Erabilitako Mekanismoak 2

3. Garatutako Ariketen Deskribapena 2

3.1. Kronometroa (Cronometroa.java) 3

3.2. Auto Lasterketa (KotxeLasterketa.java) 4

3.3. Atzerako Kontua (AtzerakoKontua.java) 5

4. Proiektua Exekutatzeko Argibideak 6

5. Ondorioak 6

0. Sarrera

Praktika honen helburua Java bidezko hari anitzeko programazioaren (Multithreading) oinarriak eta interfaze grafikoekin (Java Swing) duten elkarreragina ulertzea eta modu praktikoan garatzea da.

Mahaigaineko aplikazio grafikoetan (GUI), interfazearen gertaerak eta marrazketa hari bakar batek kudeatzen ditu: Event Dispatch Thread (EDT) izenekoak. Denbora eskatzen duen ataza bat (adibidez, segundo bat itxarotea Thread.sleep() erabiliz) hari nagusi horretan exekutatuko bagenu, interfaze osoa 'izoztu' egingo litzateke eta erabiltzaileak ezingo luke botoirik sakatu. Hori ekiditeko, atzeko planoko hari independenteak (Threads) erabiltzen dira.

Dokumentu honetan garatutako hiru ariketak, horien egitura, aplikatutako kontzeptu teorikoak eta kodearen funtzionamendua azaltzen dira.

1. Oinarri Teorikoak eta Erabilitako Mekanismoak

- **Thread eta Runnable:** Java-n hari berri bat sortzeko Runnable interfazea inplementatzen dugu eta new Thread(runnable).start() bidez abiarazten dugu. Honek sistema eragilearen hari berri bat sortzen du ataza paraleloan egiteko.
- **Interfazea ez blokeatzea:** Begizta luzeak edo denbora-itxaronaldiak (Thread.sleep(1000)) atzeko planoko hariaren barruan sartzen dira. Ondorioz, interfazearen hari nagusia libre geratzen da erabiltzailearen ekintzei (klikak, botoiak) erantzuteko.
- **Harien arteko komunikazioa eta gelditze segurua (volatile eta interrupt()):** Antzinako thread.stop() metodoa debekatuta dago arriskutsua delako. Horren ordez, kontrol-bandera boolearrak (volatile boolean martxan) eta thread.interrupt() mekanismoa erabili dira. 'volatile' hitz gakoak bermatzen du aldagai baten aldaketa hari guztiek berehala ikusten dutela memoriatik zuzenean. Haria lo dagoenean (Thread.sleep), interrupt() deitzeak InterruptedException salbuespena sortzen du, haria berehala esnatuz eta modu garbian amaituz.
- **Hari-segurtasuna eta Sinkronizazioa (synchronized):** Autoen lasterketan, 4 hari ari dira aldi berean korrika. Bi autok aldi berean helmuga zeharkatuko balute, lasterketa-baldintza (race condition) gerta liteke. Hori saihesteko, synchronized(lock) blokea erabili da: lehenengo iristen den hariak soilik eskuratzen du giltza irabazle izendatzeko.
- **SwingUtilities.invokeLater():** Swing osagaiak (adibidez progressBar.setValue() edo label.setText()) EDT haritik aldatu behar dira beti. Metodo honek atzeko planoko haritik UI-ren eguneraketak ilaran jartzen ditu modu seguruan exekutatu daitezen.

3. Garatutako Ariketen Deskribapena

Sortutako Java proiektuaren egitura Maven egitura estandarrean:

```
hariak_ariketak/
├── pom.xml
└── src/main/java/
    ├── Main.java              (Menu orokorra, 3 ariketak irekitzeko)
    ├── Cronometroa.java       (1. Ariketa: Kronometroa)
    ├── KotxeLasterketa.java   (2. Ariketa: Auto Lasterketa)
    └── AtzerakoKontua.java    (3. Ariketa: Atzerako Kontua)
```

3.1. Kronometroa (Cronometroa.java)

Ariketaren helburua: Erabiltzaileak sartutako segunduetaraino kontagailu bat aurrera eramatea. Aurrerapen-barra bat (JProgressBar) betetzen joango da eta azpiko etiketan 'X de Y segundos' erakutsiko da. Edozein unetan kontua bertan behera uzteko aukera egongo da ('Parar' botoia).

Osagaiak:

- JTextField txtSegunduak: segundo kopurua sartzeko.
- JButton btnIniciar eta JButton btnParar: kontrolerako.
- JProgressBar progressBar: 0tik N-ra betetzen doana.
- JLabel lblEgoera: segundoen aurrerapena adierazteko.

```
// Hariaren sorkuntza eta abiaraztea
kronometroHaria = new Thread(new Runnable() {
    @Override
    public void run() {
        int pasatakoSegundoak = 0;
        while (pasatakoSegundoak < segunduGuztira && martxan) {
            try {
                Thread.sleep(1000); // Segundo bat itxaron
                pasatakoSegundoak++;
                final int s = pasatakoSegundoak;
                SwingUtilities.invokeLater(() -> {
                    progressBar.setValue(s);
                    lblEgoera.setText(s + " de " + segunduGuztira + " segundos");
                });
            } catch (InterruptedException e) {
                break; // 'Parar' botoiak haria eteten du (interrupt)
            }
        }
    }
});
kronometroHaria.start();
```

3.2. Auto Lasterketa (KotxeLasterketa.java)

Ariketaren helburua: Gehienez 4 autoren arteko lasterketa simulatzea:

• Zirkuituaren distantzia konfiguragarria da (adib. 100 metro).

• Auto bakoitzaren abiadura independentea da eta erabiltzaileak doitu dezake.

• Auto bakoitza HARI INDEPENDENTE batek (Thread) mugiarazten du paraleloan.

• Auto batek helmuga zeharkatzean, irabazlea iragartzen da, beste auto guztiak berehala gelditzen dira eta bakoitzak egindako distantzia erakusten da.

```
// Auto bakoitzeko hari bana abiarazten da (4 hari paralelo)
for (int i = 0; i < AUTO_KOPURUA; i++) {
    final int autoId = i + 1;
    final double abiadura = abiadurak[i];

    Thread t = new Thread(() -> {
        double distantzia = 0.0;
        while (!lasterketaAmaituta && distantzia < helburuDistantzia) {
            try {
                Thread.sleep(100); // 100 ms-ro eguneratu kalkulua
                distantzia += abiadura * 0.1;

                if (distantzia >= helburuDistantzia) {
                    distantzia = helburuDistantzia;
                    // SINKRONIZAZIOA: Lehen iristen denak soilik izendatzen du irabazlea
                    synchronized (giltza) {
                        if (!lasterketaAmaituta) {
                            lasterketaAmaituta = true;
                            irabazlea = "Autoa " + autoId;
                        }
                    }
                }
                final double d = distantzia;
                SwingUtilities.invokeLater(() -> {
                    barrak[indizea].setValue((int) d);
                    lblDistantziak[indizea].setText(String.format("%.1f / %d m", d, (int) helburuDistantzia));
                });
            } catch (InterruptedException e) {
                break;
            }
        }
    });
    t.start();
}
```

3.3. Atzerako Kontua (AtzerakoKontua.java)

Ariketaren helburua: Erabiltzaileak programatutako segundoetatik 0ra arteko atzerako kontua egitea.

• Aurrerapen-barra hasieran balio maximoan dago eta HUSTEN joango da zeroraino.

• 0ra iristean, pantaila nagusian etiketa gorri nabarmen batekin ('¡DENBORA BUKATU DA!') eta abisu-koadro batekin (JOptionPane) jakinarazten da.

• 'Utzi' botoia sakatuz gero, kontua unean bertan bertan behera geratzen da.

```
progressBar.setMaximum(segunduHasiera);
progressBar.setValue(segunduHasiera); // Hasieran BETETA

atzerakoHaria = new Thread(() -> {
    int geratzenDena = segunduHasiera;
    while (geratzenDena > 0 && martxan) {
        try {
            Thread.sleep(1000);
            geratzenDena--;
            final int g = geratzenDena;
            SwingUtilities.invokeLater(() -> {
                progressBar.setValue(g); // Barra hustuz doa
                lblMezua.setText("Kontua martxan: " + g + " segundo geratzen dira");
            });
        } catch (InterruptedException e) {
            break;
        }
    }
    if (geratzenDena == 0 && martxan) {
        SwingUtilities.invokeLater(() -> {
            lblMezua.setText("¡DENBORA BUKATU DA! (0 segundo)");
            JOptionPane.showMessageDialog(AtzerakoKontua.this, "¡Atzerako kontua amaitu da!");
        });
    }
});
atzerakoHaria.start();
```

4. Proiektua Exekutatzeko pausoak

Proiektua bi modutara exekutatu daiteke IntelliJ IDEA-n edo edozein IDEtan:

1. Menu Nagusitik (Main.java): Main.java klasea exekutatuz gero, leiho bat irekiko da hiru botoirekin, nahi den ariketa zuzenean irekitzeko.
2. Ariketak banaka exekutatuz: Klase bakoitzak (Cronometroa, KotxeLasterketa, AtzerakoKontua) bere 'main' metodo propioa dauka, beraz, fitxategia ireki eta Run sakatuta banaka proba daitezke.

5. Ondorioak

Praktika honen bidez, honako funtsezko ezagutzak barneratu dira:

• **Interfaze grafikoak (EDT) arin eta erantzunkorrak** mantentzearen garrantzia.
• Begizta eta ataza luzeak hari laguntzaileetan (background threads) exekutatzeko beharra.
• Hariak gelditzeko eta eteteko teknika modernoak (volatile kontrol-banderak eta interrupt()).
• Hari anitzek egoera partekatuak dituztenean sinkronizazioa (synchronized) erabiltzearen ezinbestekotasuna, baldintza-lasterketak saihesteko.